pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Dates and Coupon Schedules

WEEK 2 · DAY 1

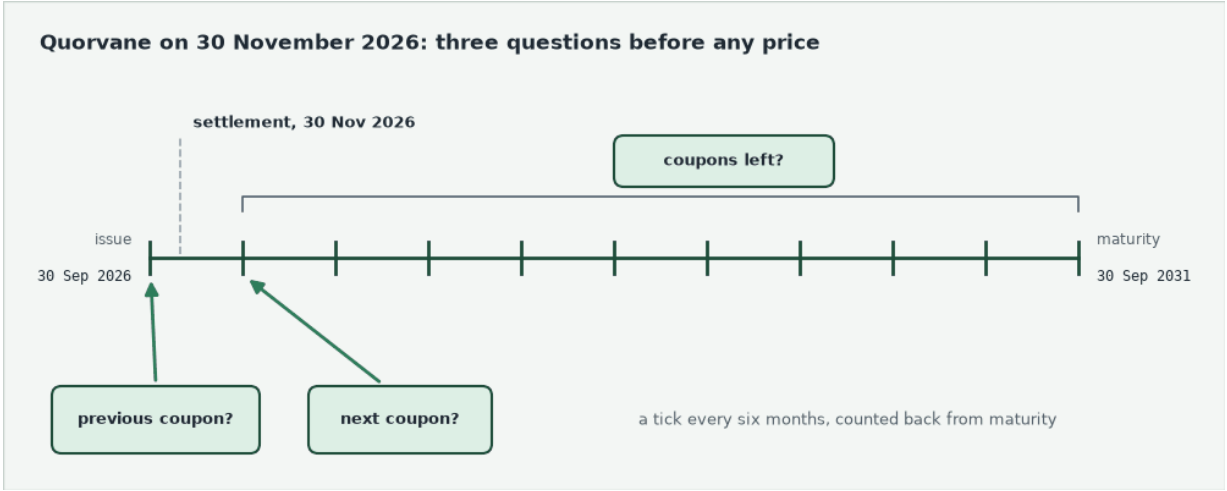
Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: three facts per bond

The Desk Lead's Note

Week 2 moves our four bonds off the coupon date: they settle Monday 30 November 2026. Before anyone prices them, I need three facts per bond: the previous coupon, the next coupon, and how many are left, each tied to Excel. Put the date logic in "bondmath", with tests. Then the holdings file: which bonds pay a coupon before year end, and how much cash is that? Last, two benchmark bonds will not tie to Excel: tell me why.



Interest accrues from the previous coupon, so every price this week starts from these three answers.

```
............   [100%]
13 passed in 0.50s
```

The module starts the day as notebook 05 left it: six functions, 13 tests.

Week 1 lived on a coupon date; today the coupon dates must be found.

6.1 Dates in Python

```
from datetime import date, timedelta
issue = date(2026, 9, 30)
settlement = date(2026, 11, 30)
print('Issue to settlement:', (settlement - issue).days, 'days')
print(settlement.year, settlement.month, settlement.day, settlement.strftime('%A'))
Issue to settlement: 61 days
2026 11 30 Monday
```

The 31 days of October and the 30 of November. The course does not move dates for weekends or holidays.

```
print('The day after:', settlement + timedelta(days=1))
quorvane_maturity = date.fromisoformat('2031-09-30')
print(settlement < quorvane_maturity, max(settlement, quorvane_maturity))
The day after: 2026-12-01
True 2031-09-30
```

A "date" prints year-month-day, the ISO layout the course's files use.

Every Maturity as a Date

The table's '2031-09-30' is ten characters of text, and Python cannot count days between two texts. Turn every maturity into a date, and check that each comes after settlement.

```
bonds['maturity_date'] = [date.fromisoformat(text) for text in bonds['maturity']]
assert (bonds['maturity_date'] > settlement).all(), 'a bond matures before settlement'
print(bonds['maturity_date'].tolist())
```

The "assert" stayed silent, and the column now holds "date" values that subtract and compare.

What Excel keeps under a date

```
excel_serial = (settlement - date(1899, 12, 30)).days
print('Excel serial of', settlement, 'is', excel_serial)
Excel serial of 2026-11-30 is 46356
```

Type "=DATE(2026,11,30)" in Excel and set the cell's number format to General: it shows 46356.

29 February 1900

```
date(1900, 2, 29)
ValueError: day is out of range for month
```

A "date" refuses a day that does not exist.

Why 30 December 1899, and not the 31st?

Excel counts 1 January 1900 as day 1 and treats 1900 as a leap year, which it was not: "=DATE(1900,2,29)" returns 60. Microsoft explains that Lotus 1-2-3 made the same assumption, and Excel kept it so worksheets could move between the two programs. That extra day is why the count in Python starts from 30 December 1899, not the 31st.

Source: Microsoft Learn, "Excel incorrectly assumes that the year 1900 is a leap year", https://learn.microsoft.com/en-us/troubleshoot/microsoft-365-apps/excel/wrongly-assumes-1900-is-leap-year

Excel returns a date as its serial number; a date format shows it as a date.

Excel side by side

Job	Python	Excel	Result
Days between	`(settlement - issue).days`	=DATE(2026,11,30)-DATE(2026,9,30)	61
Weekday	`settlement.strftime('%A')`	=TEXT(DATE(2026,11,30),"dddd")	Monday
Year, month, day	`settlement.year`	=YEAR(DATE(2026,11,30)), MONTH, DAY	2026, 11, 30
A day later	`settlement + timedelta(days=1)`	=DATE(2026,11,30)+1	46357, 1 December 2026
Earlier, later	`max(settlement, quorvane_maturity)`	=MAX(DATE(2026,11,30),DATE(2031,9,30))	48121, 30 September 2031
Text to a date	`date.fromisoformat('2031-09-30')`	=DATEVALUE("2031-09-30")	48121

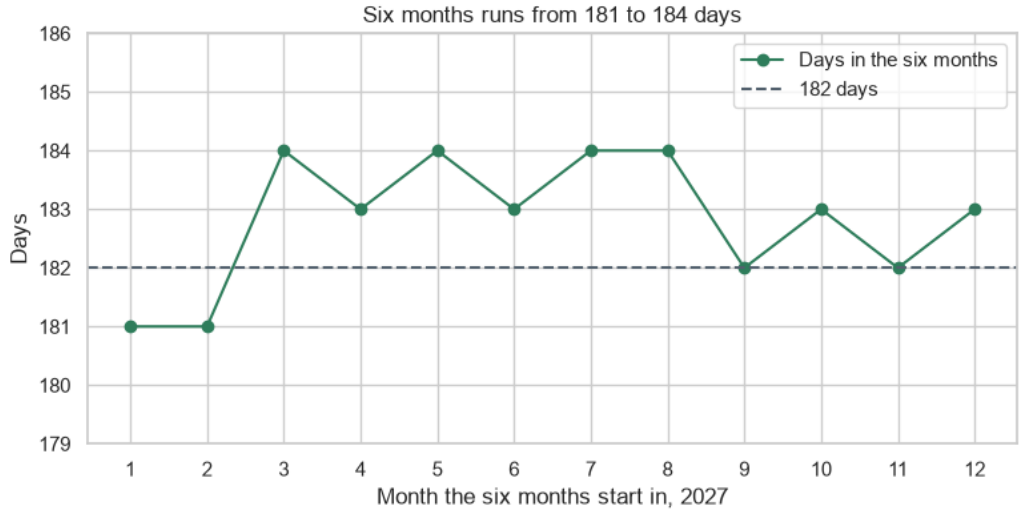
6.2 Six months is not 182 days

A semiannual bond pays every six months, and half of 365 is about 182.

```
import calendar
month_days = []
for year in [2027, 2028]:
    for month in range(1, 13):
        month_days.append(calendar.monthrange(year, month)[1])
print('February 2027:', month_days[1], ' February 2028:', month_days[13])
six_months = [sum(month_days[i:i + 6]) for i in range(12)]
print(six_months)
February 2027: 28  February 2028: 29
[181, 181, 184, 183, 184, 183, 184, 184, 182, 183, 182, 183]
```

```
date(2026, 9 + 6, 30)
ValueError: month must be in 1..12
date(2027, 2, 31)
ValueError: day is out of range for month
```

Past December the months must carry into the year, and in a short month the day must come down to its last day.

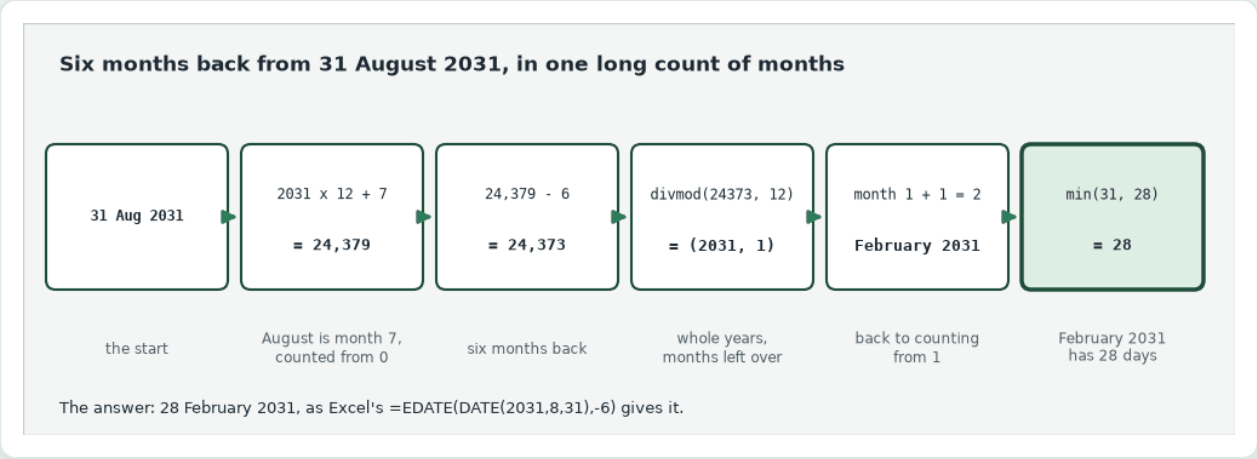


"EOMONTH(date, 0)" is the last day of the date's month, so its "DAY" is the month's length.

Job	Python	Excel	Result
Days in a month	`calendar.monthrange(2027, 2)[1]`	=DAY(EOMONTH(DATE(2027,2,1),0))	28 (2028: 29)

Moving by months must carry into the year and cut the day to a short month.

6.3 add_months: months in one long count



Number every month in one long count: the year times 12, plus the month counted from 0. Add the months, then split the count back into a year and a month.

```
month_index = 2031 * 12 + (8 - 1) - 6
year, month_zero_based = divmod(month_index, 12)
print(month_index, '->', year, month_zero_based + 1)
24373 -> 2031 2
```

Job	Python	Excel	Result
Whole years	`divmod(month_index, 12)[0]`	=QUOTIENT(2031*12+7-6,12)	2031
Months left over	`divmod(month_index, 12)[1]`	=MOD(2031*12+7-6,12)	1

```
%%writefile -a bondmath.py
import calendar
from datetime import date
def add_months(d: date, months: int) -> date:
    month_index = d.year * 12 + (d.month - 1) + months
    year, month_zero_based = divmod(month_index, 12)
    month = month_zero_based + 1
    days_in_month = calendar.monthrange(year, month)[1]
    return date(year, month, min(d.day, days_in_month))
```

"%%writefile -a" only adds to the end, so the two imports the function needs come in with it.

Five dates against EDATE, and the test

```
import importlib
importlib.reload(bondmath)
cases = [(date(2031, 8, 31), -6), (date(2032, 8, 31), -6), (date(2027, 1, 31), 1),
         (date(2026, 11, 15), 3), (date(2031, 9, 30), -6)]
for d, months in cases:
    print(d, months, '->', bondmath.add_months(d, months))
2031-08-31 -6 -> 2031-02-28
2032-08-31 -6 -> 2032-02-29
2027-01-31 1 -> 2027-02-28
2026-11-15 3 -> 2027-02-15
2031-09-30 -6 -> 2031-03-30
```

"add_months" is Excel's "EDATE", tested.

```
%%writefile -a test_bondmath.py
from datetime import date
def test_add_months_clamps_to_month_end():
    assert bondmath.add_months(date(2031, 8, 31), -6) == date(2031, 2, 28)
    assert bondmath.add_months(date(2032, 8, 31), -6) == date(2032, 2, 29)
    assert bondmath.add_months(date(2027, 1, 31), 1) == date(2027, 2, 28)
    assert bondmath.add_months(date(2026, 11, 15), 3) == date(2027, 2, 15)
```

```
..............   [100%]
14 passed in 0.50s
```

Excel side by side

Job	Python	Excel	Result
Back into a short February	`bondmath.add_months(date(2031, 8, 31), -6)`	=EDATE(DATE(2031,8,31),-6)	2031-02-28
A leap February	`bondmath.add_months(date(2032, 8, 31), -6)`	=EDATE(DATE(2032,8,31),-6)	2032-02-29
One month on from a 31st	`bondmath.add_months(date(2027, 1, 31), 1)`	=EDATE(DATE(2027,1,31),1)	2027-02-28
Across a year end	`bondmath.add_months(date(2026, 11, 15), 3)`	=EDATE(DATE(2026,11,15),3)	2027-02-15
30 September back	`bondmath.add_months(date(2031, 9, 30), -6)`	=EDATE(DATE(2031,9,30),-6)	2031-03-30

6.4 Count back from maturity

A semiannual bond pays on maturity and every six months before it, so its schedule is counted back from maturity.

Quorvane's Last Four Coupons

```
for k in range(4):
    print(k, bondmath.add_months(quorvane_maturity, -6 * k))
0 2031-09-30
1 2031-03-30
2 2030-09-30
3 2030-03-30
```

Each date is counted from maturity itself. A shorter-looking loop steps from the date before: "add_months(previous, -6)".

Is It a Month End?

```
for d in [quorvane_maturity, august_maturity, date(2031, 2, 28), date(2032, 2, 28)]:
    print(d, d.day == calendar.monthrange(d.year, d.month)[1])
2031-09-30 True
2031-08-30 False
2031-02-28 True
2032-02-28 False
```

"EOMONTH(date, months)" moves a date by whole months and returns that month's last day.

Excel side by side

Job	Python	Excel
Stepped	`bondmath.add_months(stepped[-1], -6)`	maturity in A1, =EDATE(A1,-6) in A2, filled down
Counted	`bondmath.add_months(august_maturity, -6 * k)`	1 to 4 in C2:C5, =EDATE(A1,-6*C2) in B2, filled down

Job	Python	Excel	Result
A month end, then not	`d.day == calendar.monthrange(d.year, d.month)[1]`	=DATE(2031,9,30)=EOMONTH(DATE(2031,9,30),0), then DATE(2031,8,30)	TRUE, then FALSE
February, two years	`the same, 28 February 2031 and 2032`	=DATE(2031,2,28)=EOMONTH(DATE(2031,2,28),0), then 2032	TRUE, then FALSE

coupon_dates: the previous coupon, then every one left

"coupon_dates(settlement, maturity, frequency=2)" returns the previous coupon, then every coupon left through maturity, earliest first. So "[0]" is Excel's "COUPPCD", "[1]" is "COUPNCD", and "len - 1" is "COUPNUM". It loops with "while True", which never stops on its own: "break" leaves the loop at the first date on or before settlement. "dates.reverse()" then turns the list round, earliest first.

The Loop in coupon_dates

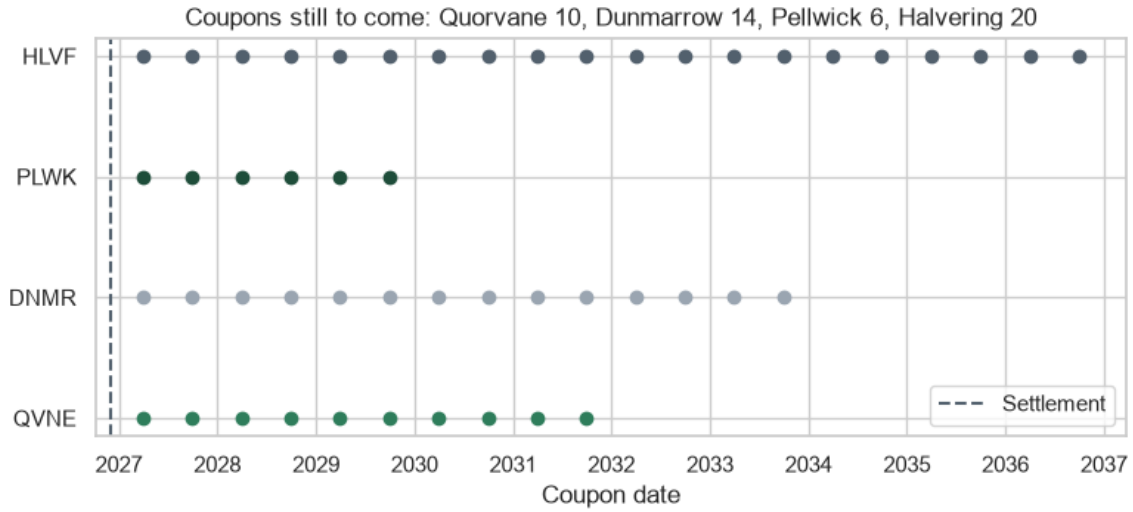
```
%%writefile -a bondmath.py
def coupon_dates(settlement: date, maturity: date, frequency: int = 2) -> list[date]:
    if frequency not in (1, 2, 4):
        raise ValueError(f"frequency must be 1, 2, or 4, not {frequency}")
    if settlement >= maturity:
        raise ValueError(f"settlement {settlement} must be before maturity {maturity}")
    step_months = 12 // frequency
    dates = []
    k = 0
    while True:
        coupon_date = add_months(maturity, -k * step_months)
        dates.append(coupon_date)
        if coupon_date <= settlement:
            break
        k += 1
    dates.reverse()
    return dates
```

Four Bonds Against Excel

ticker	previous	left
QVNE	2026-09-30	10
DNMR	2026-09-30	14
PLWK	2026-09-30	6
HLVF	2026-09-30	20

```
print('All four tie to COUPPCD, COUPNCD, and COUPNUM')
```
```
All four tie to COUPPCD, COUPNCD, and COUPNUM
```

6.6 Previous, next, and coupons left



Excel side by side

Job	Python	Excel	Result
Previous coupon	`quorvane_dates[0]`	=COUPPCD(DATE(2026,11,30),DATE(2031,9,30),2,0)	2026-09-30
Coupons left	`len(quorvane_dates) - 1`	=COUPNUM(DATE(2026,11,30),DATE(2031,9,30),2,0)	10

The last argument, the basis, is the day count (0 is 30/360), and it does not move the dates.

The test on 306 rows, and a .gitignore

```
bondmath.coupon_dates(date(2031, 10, 1), quorvane_maturity)
ValueError: settlement 2031-10-01
must be before maturity 2031-09-30
```

A matured bond has no next coupon, and a silent answer would be wrong. The test runs "coupon_dates" on all 306 rows of Excel's file, 50 edge cases among them.

```
%%writefile -a test_bondmath.py
def test_coupon_dates_match_excel():
    for row in REFERENCE:
        settlement, maturity = date.fromisoformat(row["settlement"]), date.fromisoformat(row["maturity"])
        dates = bondmath.coupon_dates(settlement, maturity, int(row["frequency"]))
        assert dates[0] == date.fromisoformat(row["couppcd"]), row["case_id"]
        assert dates[1] == date.fromisoformat(row["coupncd"]), row["case_id"]
        assert len(dates) - 1 == int(row["coupnum"]), row["case_id"]
```

```
...............   [100%]
15 passed in 0.50s
```

"__pycache__/" is where Python keeps compiled copies of the module and the tests. Python rebuilds it at will, so it never belongs in the record. A ".gitignore" file lists names Git should not offer to track, one to a line; a name ending in "/" is a folder.

```
%%writefile .gitignore
# folders Python and pytest make: never part of the record
__pycache__/
.pytest_cache/
```

```
 M bondmath.py
 M test_bondmath.py
?? .gitignore
e169ff0 (HEAD -> main) Notebook 06: add_months, coupon_dates, and a .gitignore
322960d (tag: week1) Notebook 05: tests of par pricing and of price against yield
```

Git records the work, not what Python rebuilds.

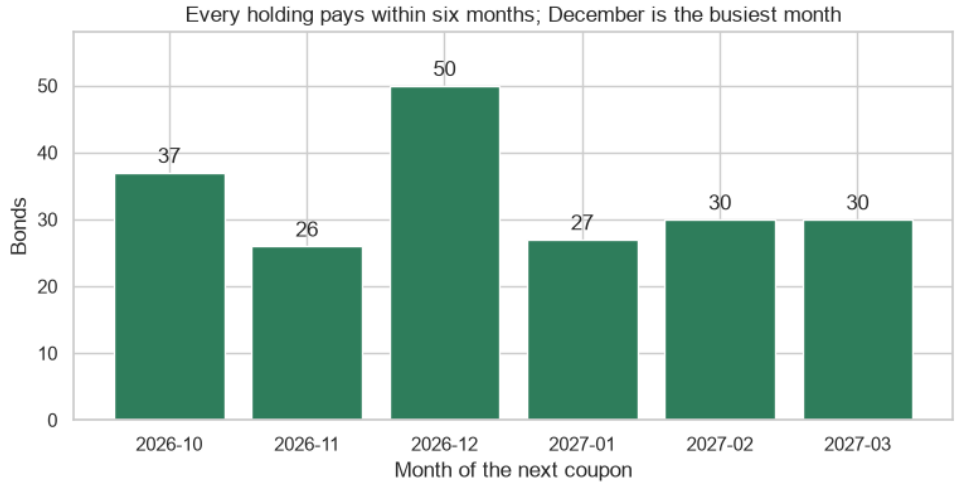
6.8 The holdings' coupon calendar

```
holdings = pd.read_csv('holdings.csv')
maturities = pd.to_datetime(holdings['maturity'])
200 bonds, 0 maturing on a month end
```

```
as_of = pd.to_datetime(holdings['as_of_date']).dt.date
holdings['next_coupon'] = [bondmath.coupon_dates(a, m)[1]
                           for a, m in zip(as_of, maturities.dt.date)]
```

cusip	as_of_date	maturity	next_coupon
99080CAE8	2026-09-30	2032-06-18	2026-12-18
99049XAE2	2026-09-30	2029-04-17	2026-10-17
99039NAC0	2026-09-30	2030-06-03	2026-12-03

coupon_month	bonds	dollars
2026-10	37	2,930,468.75
2026-11	26	1,833,906.25
2026-12	50	3,776,406.25




```
due = holdings[holdings['next_coupon'] <= date(2026, 12, 31)].copy()
due['coupon_dollars'] = due['par_held'] * due['coupon'] / 100 / 2
113 bonds pay 8,540,781.25 dollars by year end
```

This describes the file's cash calendar; it says nothing about any bond's value.

Bring it home, and what you can now do

Copy the work folder's three files into your own folder, then test and commit there. In a PowerShell terminal in VS Code, with the course's ".venv" active, use the work folder name your setup cell printed:

```
Set-Location "$HOME\projects\my_bondmath"
$work = "$env:TEMP\pfi_course2_06_k4w7q2xr"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py", "$work\.gitignore" .
python -m pytest -q
git status --short
git add bondmath.py test_bondmath.py .gitignore
git commit -m "Notebook 06: add_months, coupon_dates, and a .gitignore"
```

The copy replaces your files, so anything of your own goes into the work folder first. In Colab, download the files instead. The work folder stays in your temp folder until you delete it.

You Can Now

Make, compare, and subtract dates, and say what Excel's date serials count.

Move a date by months with "add_months" or "EDATE".

Apply the month-end rule, with "EOMONTH" in Excel.

Find the previous coupon, next coupon, and coupons left with "coupon_dates", tied to Excel.

Hand a pandas date column to the module with ".dt.date".

Keep the cache out of Git with a ".gitignore".

Predict first: 182 days, stepped

Predict 1

The second coupon after issue falls on 30 September 2027. Step 182 days from issue, then 182 more. Where do you land?

A) 30 September 2027, on the coupon date

B) 29 September 2027, a day short

C) 1 October 2027, a day past

```
first = issue + timedelta(days=182)
print(issue, '+ 182 days =', first)
print(first, '+ 182 days =', first + timedelta(days=182))
```

Predict 2

The tenth coupon after issue is Quorvane's maturity, 30 September 2031. Ten steps of 182 days from issue: how many days short of maturity do you land? Write down a number.

```
tenth = issue + timedelta(days=182 * 10)
print(tenth, 'is', (quorvane_maturity - tenth).days, 'days short of', quorvane_maturity)
```

Predict first: counted from maturity, and Excel's rule

Predict 3

A bond matures on 30 August 2031. February has no 30th, so the first date back is 28 February 2031 either way. Over the first four dates back, on how many do stepping and counting differ?

A) None

B) One, August 2030

C) Two, both Augusts

```
august_maturity = date(2031, 8, 30)
stepped = [august_maturity]
for k in range(4):
    stepped.append(bondmath.add_months(stepped[-1], -6))
counted = [bondmath.add_months(august_maturity, -6 * k) for k in range(1, 5)]
pd.DataFrame({'stepped': stepped[1:], 'counted': counted})
```

Predict 4

Two bonds: Quorvane, maturing 30 September 2031, and a bond in Excel's reference file maturing 30 March 2031, settled 15 October 2026. Counted from maturity, both next coupons fall on 30 March 2027. What does Excel's "COUPNCD" say?

A) Quorvane moves to 31 March 2027; the other bond stays on the 30th

B) Both move to 31 March 2027

C) Neither moves

```
reference = pd.read_csv('bondmath_excel_reference.csv', index_col='case_id')
print('Quorvane       Excel', reference.loc['99001BZA7', 'coupncd'],
      ' counted', bondmath.add_months(quorvane_maturity, -6 * 9))
print('30 March bond  Excel', reference.loc['edge_mat_30_mar', 'coupncd'],
      ' counted', bondmath.add_months(date(2031, 3, 30), -6 * 8))
```

The note, answered

The note asked	Answer
Previous coupon	30 September 2026, all four
Coupons left	Quorvane 10, Dunmarrow 14, Pellwick 6, Halvering 20
The date logic, with tests	add_months and coupon_dates,
	15 tests, all 306 rows of COUPPCD, COUPNCD, and COUPNUM
Coupon cash by year end	113 bonds, 8,540,781.25 dollars on the par held

Be proud: your schedule is right to the day, and you can prove it on every row Excel gave you.

Next, Day Counts and Accrued Interest (notebook 07) counts the days between these dates.